

RAPPORT TECHNIQUE

Moteur Morphologique Arabe

Projet : Mini-projet Algorithmique

Année : 2025-2026

Responsables : Narjes Ben Hariz, Sahbi Bahroun

Réalisatrices : Sebai Arij, Hanafi Balkis, Hosni Hadil

Langage : C++17

1. STRUCTURES DE DONNÉES UTILISÉES

1.1 Arbre AVL (Racines)

Un **arbre AVL auto-équilibré** stocke les racines trilitères arabes avec deux propriétés clés :

- **Équilibrage** : différence hauteur gauche/droite ≤ 1
- **Cache intégré** : chaque nœud maintient les dérivés générés

Structure : Nœud = (racine, hauteur, dérivés_cachés, map<fréquences>)

1.2 Table de Hachage (Schèmes)

Implémentation manuelle avec **chaînage pour collisions** :

- **Fonction** : djb2 (algorithme $33x+c$) offrant distribution uniforme sur UTF-8 arabe
- **Redimensionnement** : automatique quand facteur charge > 0.75
- **Avantage** : recherche $O(1)$ pour 14 schèmes critiques

1.3 Index Inversé (Validation rapide)

Pré-calcul au démarrage : **map<mot_simplifié, (racine, schème)>**

- **700 entrées** pour $50 \text{ racines} \times 14 \text{ schèmes} = \sim 28 \text{ KB}$
- Réduit vérification de $O(n \times k \times m)$ à **$O(1)$**

1.4 Caches Auxiliaires

- Cache simplification : évite recalcul diacritiques ($O(1)$ au lieu $O(m)$)
 - Dérivés en nœuds AVL : évite régénération répétée
-

2. ALGORITHMES : GÉNÉRATION ET VALIDATION

2.1 Génération Morphologique ($O(k)$)

Système racine-schème : insertion consonantique dans patron abstrait

1. **Extraction caractères** : racine "س, ل, ج" $\rightarrow$ "جلس"

- 2. **Application schème** : "فاعل" + marqueurs temporaires (\x01, \x02, \x03) pour éviter remplacement multiple
- 3. **Résultat** : "جالس" (dérivé valide)

Itération sur 14 schèmes → **14 dérivés par racine**

2.2 Validation Morphologique (O(1) optimisé)

Entrée : mot à vérifier

1. Simplifier (cache) : O(1)

2. Chercher index inversé : O(1)

3. Retourner (racine, schème) si trouvé

Alternative naïve aurait été : boucle sur 50 racines × 14 schèmes × génération = **3500 opérations**

3. JUSTIFICATION DES CHOIX ALGORITHMIQUES

Structure	Choix	Justification
AVL	vs ABR binaire	O(log n) garanti vs O(n) pire cas; affichage alphabétique exigé
HashTable	vs vecteur/arbre	O(1) vs O(log 14)=3.8 pour accès fréquent schèmes
Index inversé	Trade-off mémoire	Gain temps 700× (28KB mémoire acceptable)
djb2	vs FNV-1a/Murmur	Simplicité + distribution excellente sur UTF-8

Exemple gain AVL : Pour 50 racines : log₂(50) ≈ **6 comparaisons** vs 50 en pire cas

4. COMPLEXITÉ ALGORITHMIQUE

4.1 Opérations temporelles

Arbre AVL :

- Insertion/Recherche : **O(log n)** = ~6 pour n=50
- Parcours complet : **O(n)**

Table Hachage :

- Insertion/Recherche moyenne : **O(1)** (facteur charge < 0.75)
- Pire cas chaîné : **O(α)** où α = nombre éléments/buckets

Validation morphologique :

- Initiale : O(n×k×m) = O(50×14×5) = 3500 opérations
- Optimisée : **O(1)** avec index inversé

4.2 Espace mémoire

- Arbre AVL : 50×150 bytes ≈ 7.5 KB
- HashTable : 14×80 bytes ≈ 1.1 KB
- Index inversé : 700×40 bytes ≈ 28 KB
- **Total : ~40 KB** (négligeable pour performance)

5. PRINCIPALES DIFFICULTÉS RENCONTRÉES

5.1 Encodage UTF-8 Arabe

Problème : Caractères arabes = 3 octets UTF-8 ; indexation naïve `string[i]` corrompt les données

Solution : Parser UTF-8 manuel détectant début caractère (bits 11xxxxxx)

```
if((c & 0xF0) == 0xE0) len = 3; // Arabe
```

Impact : A retardé phase initiale mais solidifié compréhension encodage

5.2 Performance Vérification Morphologique

Problème initial : ~500ms/mot pour 50 racines (complexité $O(n \times k \times m)$)

Solutions progressives :

1. Cache dérivés → **gain 14×**
2. Index inversé → **gain 700×**
3. Cache simplification → **gain 9×**

Résultat : < 1ms/mot (gain total ~500×)

5.3 Verbes Irréguliers Arabes

Problème : Mots comme "قرأ", "أخذ", "قال" violent règles standards

Solution : Map séparée `verbes_irreguliers` consultée prioritairement dans génération

CONCLUSION

Implémentation complète moteur morphologique arabe combinant structures optimales (AVL + HashTable personnalisée) avec algorithmes adaptés (index inversé, caches) atteignant performances constantes $O(1)$ sur opération critique (validation). Gestion UTF-8 correcte garantit robustesse multilingue.